



React Native — Développer une application mobile multiplateforme

Formation intensive · 3 jours · Exécution 100 % navigateur (Expo Web) — aucun émulateur requis

NIVEAU DÉBUTANT À INTERMÉDIAIRE

EXPO SDK 57 · REACT NATIVE 0.86

Comment lire ce support

Ce support est organisé en six parties progressives. Chaque partie s'appuie sur la précédente pour construire une compétence complète du développement mobile multiplateforme.



Partie 1 — Fondations et écosystème

Comprendre React Native, son histoire et la place d'Expo



Partie 2 — Socle technique

Composants, style, listes, formulaires



Partie 3 — Navigation, données, état

Routing, API, persistance locale



Parties 4, 5 & 6 — Avancé, IA et projet

Animations, bonnes pratiques IA, soutenance



Les blocs EX-xx renvoient au fichier `02-exemples-code.md` fourni dans le dépôt de la formation.

Objectifs pédagogiques

À l'issue de ces 3 jours, vous serez autonome sur les briques fondamentales du développement mobile avec React Native et Expo.



Expliquer l'architecture

Comprendre le fonctionnement interne de React Native et la valeur ajoutée d'Expo



Construire une interface native

Maîtriser les composants de base, le style Flexbox et la gestion des listes



Naviguer entre les écrans

Structurer une application multi-écrans avec Expo Router



Consommer des données

Appeler une API distante, gérer les états de chargement, persister localement



Livrer et présenter

Déployer, rédiger un README, et soutenir son projet face au jury

Déroulé des 3 jours

Moment	Contenu
J1 matin	Cours magistral + démos guidées — fondations, écosystème, mise en place
J1 après-midi	Cadrage du projet, maquettes, initialisation de l'environnement
J2	Développement en autonomie, support à la demande, revues de code
J3 matin	Finalisation, README, préparation de la démo
J3 après-midi	Soutenances — 10 min de démo + 5 min de questions



PARTIE 1 — FONDATIONS

Comprendre React Native

Ce qu'il est, ce qu'il n'est pas, et pourquoi on l'utilise en 2025.

Le problème de départ

Développer pour mobile, c'est historiquement choisir son camp — et payer le prix fort en duplication d'effort.



iOS

Swift / Objective-C, Xcode, un Mac obligatoire. Un écosystème fermé avec ses propres règles.



Android

Kotlin / Java, Android Studio. Un autre écosystème, d'autres outils, d'autres bugs.

Le coût réel

Deux bases de code, deux équipes, deux fois les bugs, deux fois les mises à jour.

Question posée depuis 2010 : peut-on écrire **une seule fois** et livrer **partout** ?

Les trois familles de réponses

1

Web mobile / PWA

Un site optimisé mobile. Simple à mettre en œuvre, mais accès limité au matériel, pas de distribution via les stores.

2

WebView hybride

Cordova, Ionic — une page web emballée dans une app. Rendu non natif, performances souvent décevantes.

3

Compilation vers le natif

React Native, Flutter, .NET MAUI. Le rendu est réellement natif, les performances et l'UX sont au niveau des apps système.



React Native en une phrase

Vous écrivez du **JavaScript / React**, et le framework pilote de **vrais composants natifs** iOS et Android.

Votre code

`<View>` — un composant React familier

Ce que voit l'utilisateur

`UIView` sur iOS · `ViewGroup` sur Android. Pas de `WebView`, pas de compromis.

Ce que React Native n'est pas

✗ Une WebView déguisée

Les composants sont rendus par les moteurs natifs de chaque plateforme, pas par un navigateur embarqué.

✗ Du HTML pour mobile

Aucune balise HTML n'arrive jusqu'à l'écran. `<View>` n'est pas une `<div>` sur iOS.

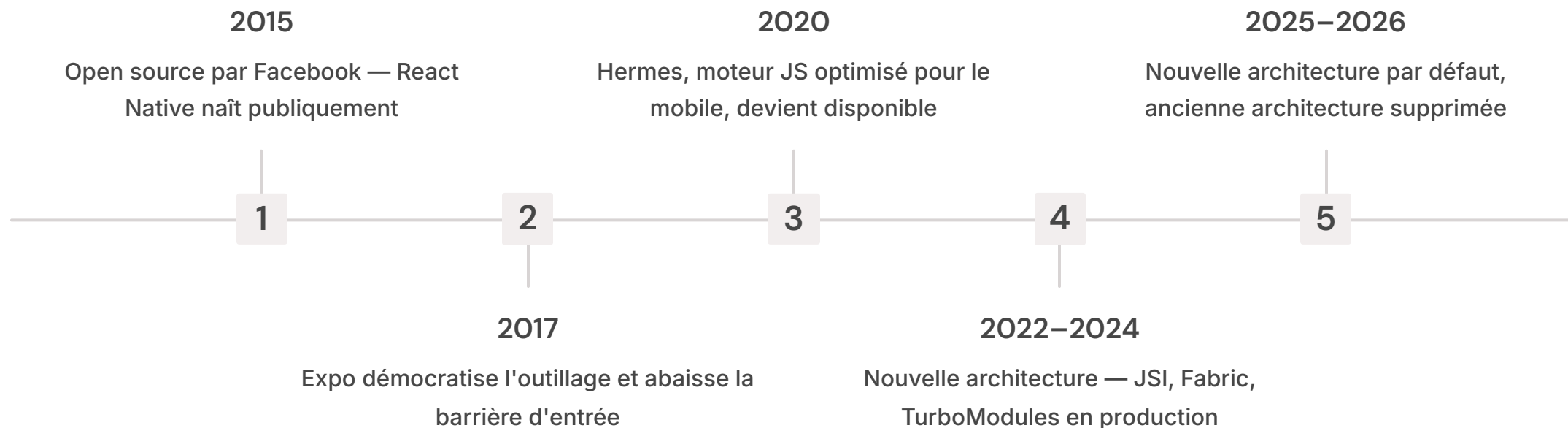
✗ Une solution magique

Des différences de comportement par plateforme existent et doivent être gérées explicitement.

✓ Du React, autrement

Même modèle mental, même syntaxe JSX, mêmes hooks — mais un arbre de rendu différent du DOM.

Un peu d'histoire



Qui l'utilise ?

Des entreprises parmi les plus exigeantes en matière de performance mobile font confiance à React Native en production.

Instagram

Discord

Shopify

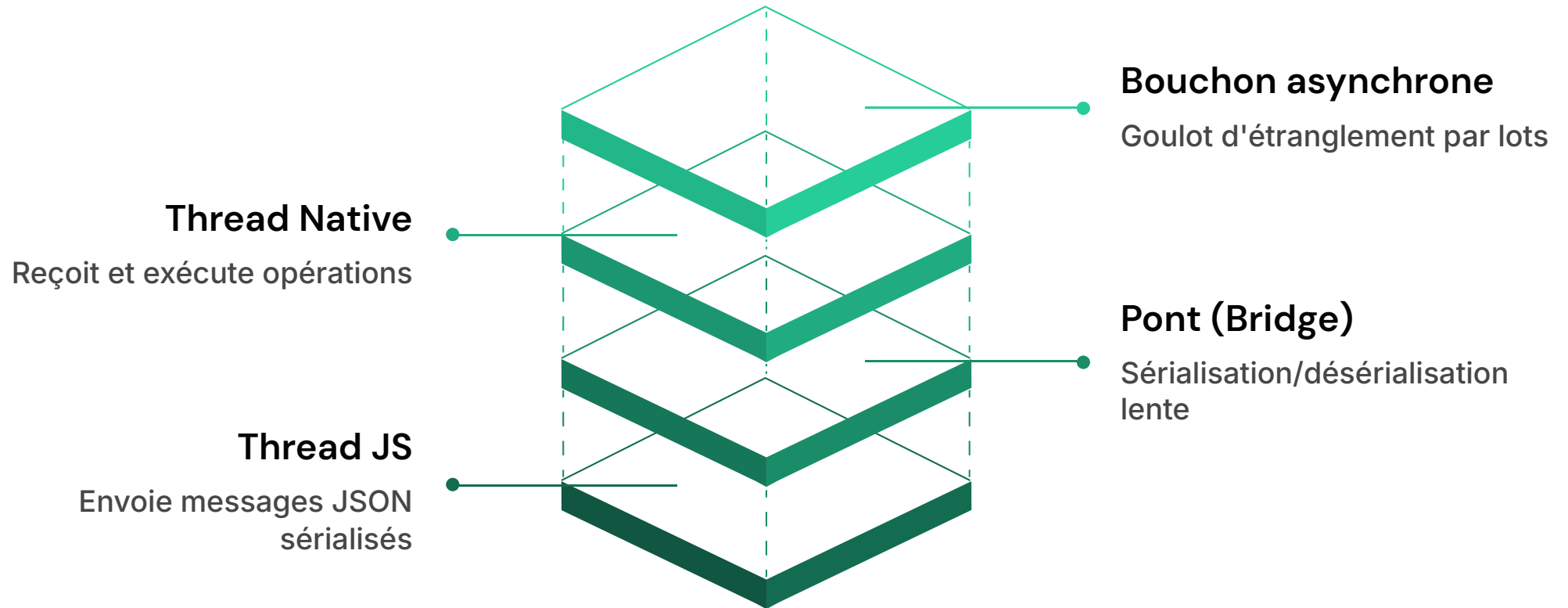
Microsoft Office

Coinbase

Bloomberg

 Souvent en **mode hybride** : une app native historique avec des écrans React Native intégrés. Rarement du 100 % RN sur les très grosses apps.

L'ancienne architecture : le Bridge



Comment ça fonctionnait

Toute communication entre le code JavaScript et les composants natifs transitait par un pont (*bridge*), en sérialisant les données en JSON.

Les limites

- Communication asynchrone et par lots
- Tout passe par la sérialisation JSON
- Goulot d'étranglement sur les animations et les longues listes

La nouvelle architecture



JSI — JavaScript Interface

Le JS appelle directement des fonctions C++ sans sérialisation.
Plus de pont, des appels synchrones possibles.



Fabric — Nouveau moteur de rendu

Rendu synchrone possible, meilleure gestion des mises à jour concurrentes avec React 18+.



TurboModules

Modules natifs chargés à la demande. Démarrage plus rapide, empreinte mémoire réduite.



Codegen

Génération de code typé à partir de spécifications TypeScript.
Moins d'erreurs à l'interface JS/natif.

Hermes : le moteur JavaScript de Meta

Conçu pour le mobile

Hermes est le moteur JavaScript par défaut de React Native, taillé pour les contraintes des appareils mobiles — batterie, mémoire, vitesse de démarrage.

- Compilation **AOT** en bytecode → démarrage plus rapide
- Empreinte mémoire significativement réduite
- Débogage via React Native DevTools



React Native vs Flutter

Critère	React Native	Flutter
Langage	JS / TypeScript	Dart
Rendu	Composants natifs réels	Canvas propre (Skia/Impeller)
Look	Natif par plateforme	Identique partout
Écosystème	npm entier	pub.dev
Courbe	Douce si on connaît React	Nouveau langage à apprendre

📌 Pas de « meilleur » absolu : le bon choix dépend avant tout de l'équipe existante et de ses compétences.

Où se place Expo ?

React Native seul = le moteur

Puissant, mais nu. Vous configurez tout vous-même : bundler, liens natifs, builds, certificats...

Expo = la voiture complète

- CLI, serveur de dev, bundler préconfigurés
- Bibliothèque de modules natifs prêts à l'emploi
- Expo Router pour la navigation
- EAS pour les builds cloud et les mises à jour à distance
- **Support web via react-native-web**

Versions de référence

SDK 57

Expo

Juin 2026 — version utilisée dans ce support

0.86

React Native

Nouvelle architecture activée par défaut

19.2

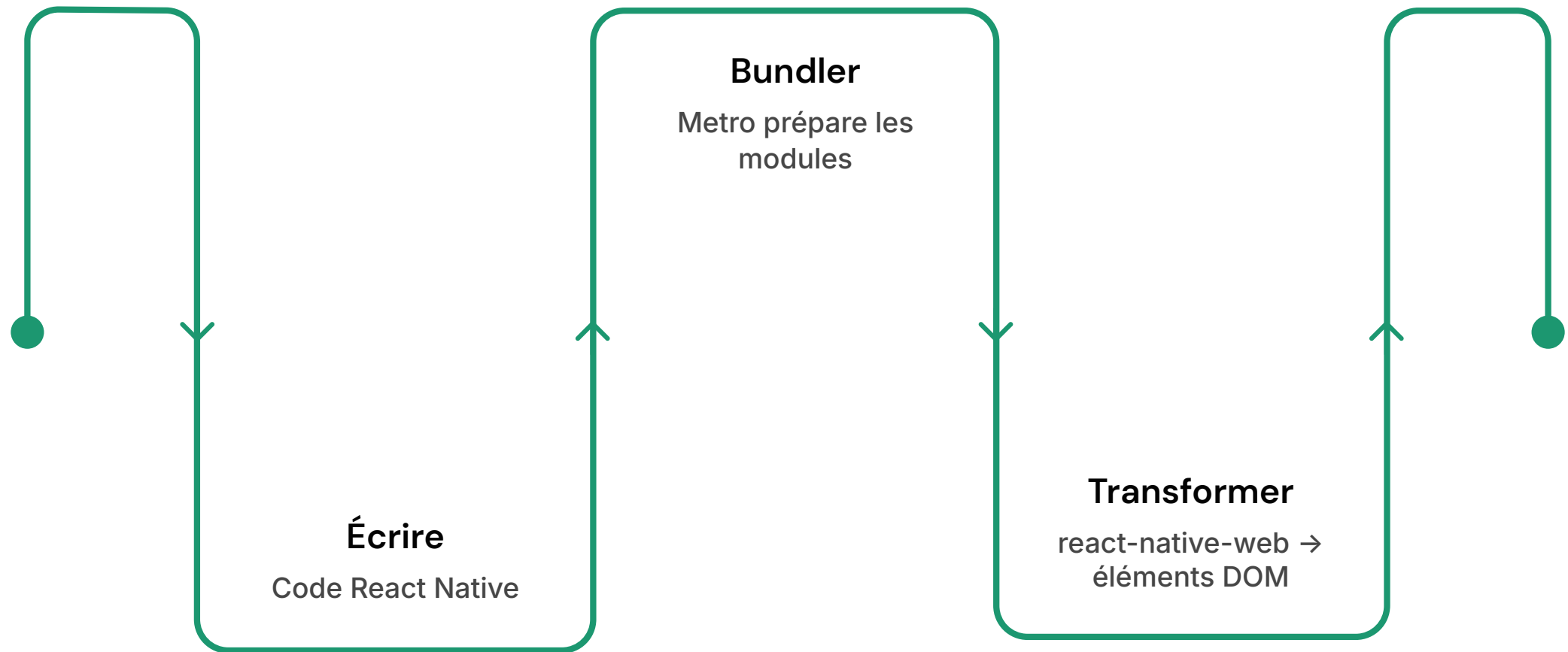
React

Concurrent features, Server Components

 L'écosystème bouge vite : vérifiez toujours **docs.expo.dev** et le changelog de la version que vous installez.

Notre contrainte : le navigateur

Pas d'émulateur, pas de Mac, pas de téléphone requis. Nous exécutons tout via **react-native-web**.



Ce qui se passe

```
<View> → <div>  
<Text> → <span>  
<Image> → <img>
```

Le résultat

Une vraie application React Native dans un onglet de navigateur, avec hot reload et DevTools.

Ce qui fonctionne bien en mode web

View & Text

Image & ScrollView

FlatList

Pressable & TextInput

Modal

StyleSheet & Flexbox

Animated

Navigation Expo Router

fetch & AsyncStorage

SVG, icônes, polices

Ce qu'il faut éviter en mode web

Alert.alert()

Sans effet sur le web. Remplacez par un composant `Modal` ou `window.confirm`.

Caméra, capteurs, Bluetooth, NFC

APIs matérielles non accessibles depuis le navigateur. À exclure du périmètre projet.

Notifications push natives

Non supportées en mode web sans Service Worker et configuration spécifique.

Haptics, biométrie, elevation

Ignorés silencieusement ou produisent des erreurs. `KeyboardAvoidingView` également.

📋 Consigne projet : concevez pour ce périmètre. Toutes les fonctionnalités attendues sont réalisables sans ces APIs.

La règle d'or du cross-platform

Certaines différences de comportement entre plateformes sont inévitables. React Native propose deux approches pour les gérer proprement.

Platform.select() — inline

```
import { Platform } from 'react-native';

const espacement = Platform.select({
  ios: 20,
  android: 16,
  web: 24,
});
```

Fichiers séparés par plateforme

```
Bouton.ios.tsx
Bouton.android.tsx
Bouton.web.tsx
```

Metro charge automatiquement le bon fichier selon la cible. Idéal pour des différences plus importantes.

Mise en place de l'environnement

Quatre commandes suffisent pour avoir une application React Native qui tourne dans le navigateur.

```
node --version      # 20 LTS ou plus

npx create-expo-app@latest mon-app
cd mon-app
npx expo install react-dom react-native-web @expo/metro-runtime

npx expo start --web
```

L'application s'ouvre sur `http://localhost:8081`. Le hot reload est actif dès le premier enregistrement.

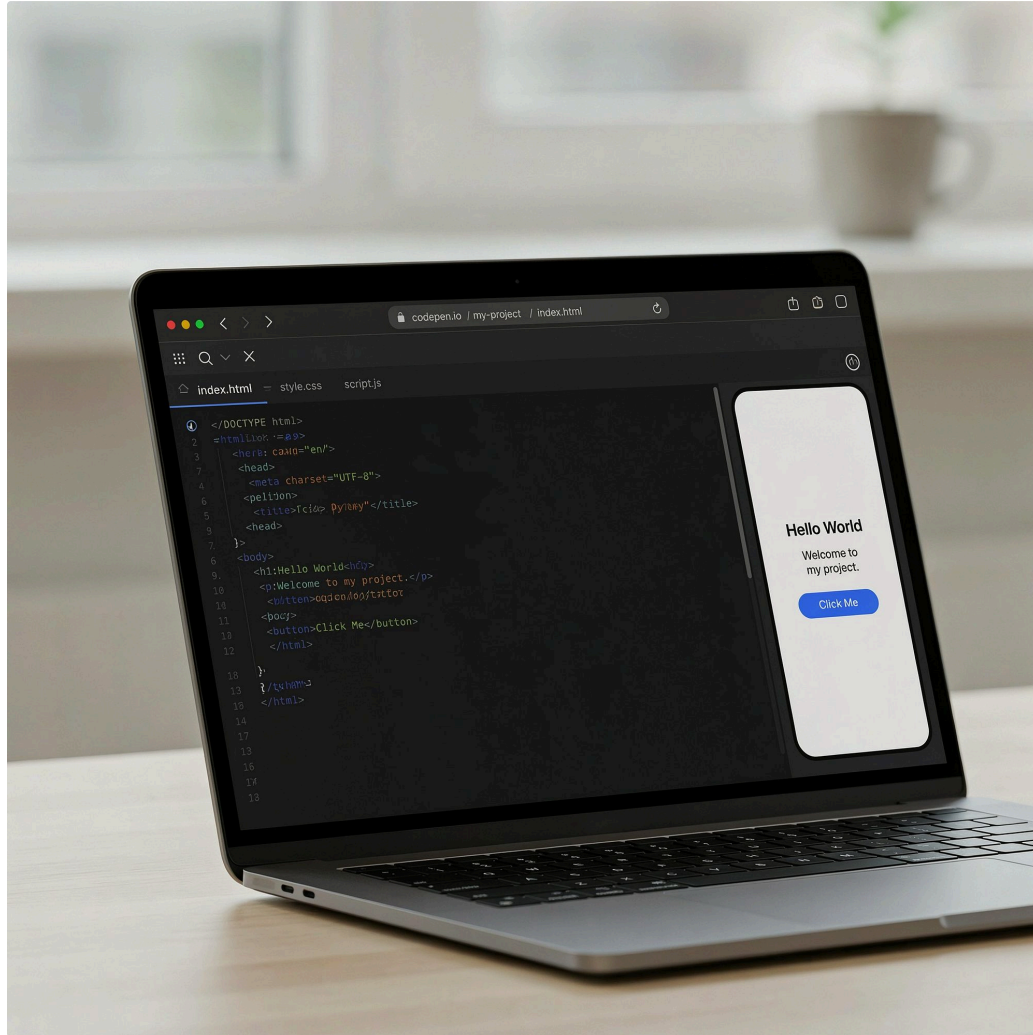
Anatomie d'un projet Expo

```
mon-app/  
├─ app/      écrans (routing par fichiers)  
│  ├─ _layout.tsx  layout racine  
│  └─ index.tsx    écran d'accueil "/"  
├─ assets/      images, polices  
├─ components/   composants réutilisables  
├─ app.json      configuration de l'app  
├─ package.json  
└─ tsconfig.json
```

Le principe clé

Avec Expo Router, l'**arborescence du dossier**

Le plan B : Expo Snack



snack.expo.dev

Éditeur en ligne avec aperçu web instantané et aucune installation requise.

Idéal pour...

- Démarrer en 10 secondes
- Partager un extrait de code en formation
- Dépanner un poste récalcitrant



Limite : peu adapté à un projet de plusieurs jours avec plusieurs fichiers et dépendances complexes.

PARTIE 2 — SOCLE TECHNIQUE

Composants, style, listes, saisie

Les briques fondamentales pour construire n'importe quelle interface mobile.



Le premier écran

```
import { View, Text, StyleSheet } from 'react-native';

export default function Accueil() {
  return (
    <View style={styles.conteneur}>
    <Text style={styles.titre}>Bonjour
```

Ce qui change par rapport au web

Web HTML	React Native
<code><div></code>	<code><View></code>
<code><p></code> , <code></code> , <code><h1></code>	<code><Text></code>
<code></code>	<code><Image></code>
<code><input></code>	<code><TextInput></code>
<code><button></code> , <code>onClick</code>	<code><Pressable></code> , <code>onPress</code>
CSS, classes	Objets JavaScript
Scroll implicite	<code><ScrollView></code> explicite

Règle n°1 : tout texte dans un Text

✗ Erreur classique

```
<View>Bonjour</View>
```

Génère une erreur de rendu. C'est l'erreur n°1 des développeurs web qui débutent en React Native.

✓ La bonne façon

```
<View>  
  <Text>Bonjour</Text>  
</View>
```

Tout contenu textuel, sans exception, doit être enveloppé dans un composant Text.

View : la brique de base

Conteneur universel

Équivalent d'une `div`, mais sans aucun style par défaut. Pas de marge, pas de padding, fond transparent.

Flexbox par défaut

Tout `View` est un conteneur Flexbox avec `flexDirection: 'column'`. Pas besoin de le déclarer.

Ne peut pas contenir de texte brut

Jamais de texte directement dans un `View` — toujours enveloppé dans `Text`.

```
<View style={{ padding: 16, backgroundColor: '#eee' }}>
  {/* enfants */}
</View>
```

Text : composition et héritage

```
<Text style={{ fontSize: 16 }}>
  Bonjour{' '}
  <Text style={{ fontWeight: 'bold' }}>
    Marc
  </Text>
  , bienvenue.
</Text>
```

Ce qu'il faut savoir

- Les Text imbriqués **héritent** du style parent
- `numberOfLines={2}` pour tronquer avec ellipse
- `onPress` fonctionne directement sur un Text
- `selectable` pour permettre la sélection du texte

Image : local et distant

Ressource locale

```
<Image  
  source={require('./assets/logo.png')}  
  style={{ width: 80, height: 80 }}  
>
```

Ressource distante

```
<Image  
  source={{ uri: 'https://picsum.photos/300' }}  
  style={{ width: 300, height: 200 }}  
  resizeMode="cover"  
>
```

⚠ Sans `width` et `height` explicites, une image distante ne s'affiche pas. Les dimensions sont **toujours obligatoires**.

expo-image : l'alternative moderne

Recommandée pour la production

```
import { Image } from 'expo-image';

<Image
  source="https://picsum.photos/300"
  style={{ width: 300, height: 200 }}
  contentFit="cover"
  transition={200}
  placeholder={flouBase64}
/>
```

Avantages

- Cache disque et mémoire automatique
- Transitions animées
- Placeholder flou (blurhash)
- Formats modernes (AVIF, WebP)
- Performances supérieures sur mobile

ScrollView

```
<ScrollView  
  contentTypeStyle={{ padding: 16 }}  
>  
  { /* contenu long */ }  
</ScrollView>
```

Les deux attributs de style

- `style` → stylise le **conteneur visible** (bordure, fond, dimensions)
- `contentTypeStyle` → stylise le **contenu interne** (padding, alignement)

⚠️ ScrollView rend **tous** ses enfants d'un coup. Ne l'utilisez jamais pour de longues listes — utilisez FlatList.

Pressable : interactions tactiles

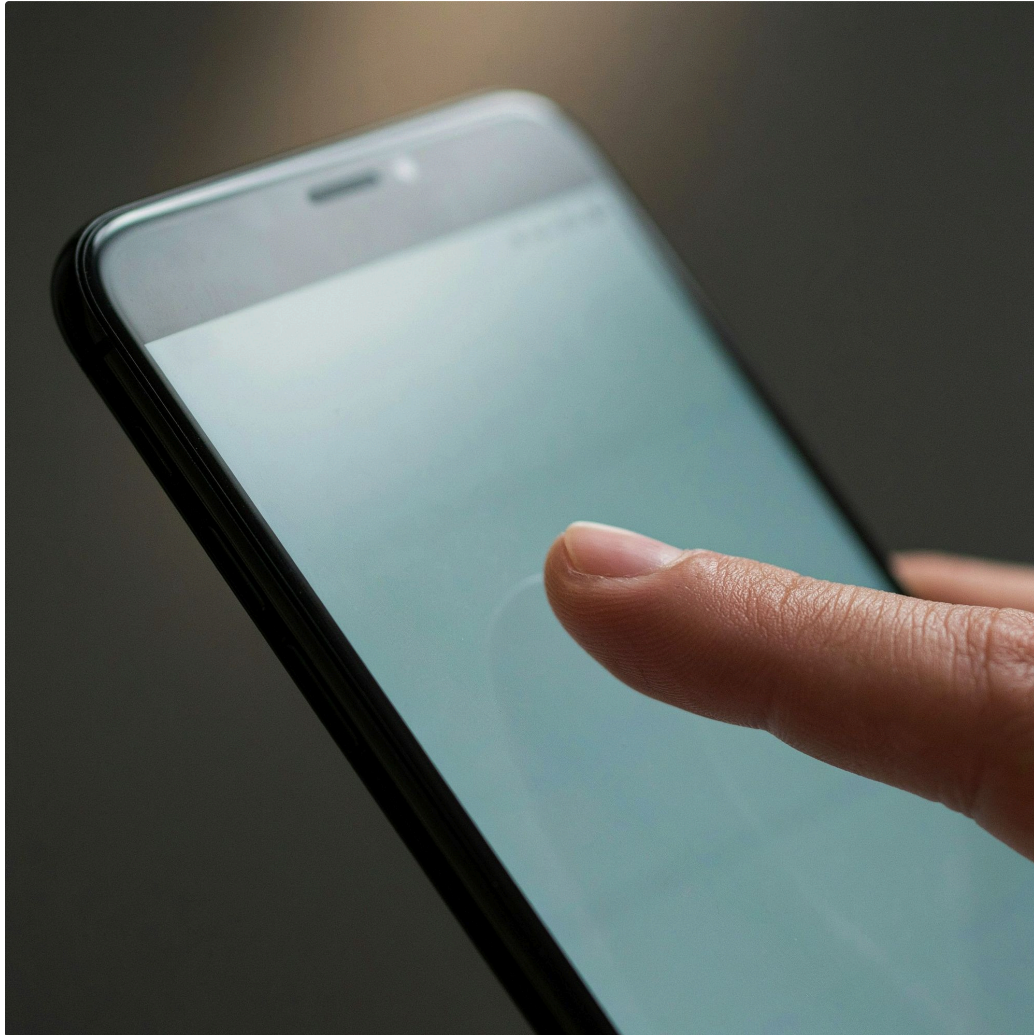
```
<Pressable
  onPress={() => console.log('clac')}
  onLongPress={() => console.log('appui long')}
  style={({ pressed }) => [
    styles.bouton,
    pressed && { opacity: 0.6 },
  ]}
>
  <Text style={styles.libelle}>Valider</Text>
</Pressable>
```

Pourquoi Pressable ?

Composant	Usage
Pressable	Le choix par défaut aujourd'hui
TouchableOpacity	Ancien, atténue l'opacité
Button	Très limité, non stylisable

 Voir EX-04 pour un bouton réutilisable complet.

La zone tactile minimale



Recommandation Apple & Google

La zone touchable minimale est de **44 × 44 points**. En dessous, l'utilisateur rate la cible et s'exaspère.

```
<Pressable  
  hitSlop={8}  
  style={{ minHeight: 44, minWidth: 44 }}  
>
```

hitSlop étend la zone sensible sans modifier le visuel — idéal pour les petites icônes.

SafeAreaView

Sur les appareils physiques, l'interface est rogée par les encoches, la barre d'état et les barres de gestes. SafeAreaView gère tout ça automatiquement.

```
import { SafeAreaView } from 'react-native-safe-area-context';

<SafeAreaView style={{ flex: 1 }} edges={['top', 'bottom']}>
  {/* contenu */}
</SafeAreaView>
```

 Sans effet visible sur navigateur, mais indispensable dès que vous testez sur un vrai téléphone via Expo Go. Prenez l'habitude dès maintenant.

Le style en React Native

```
const styles = StyleSheet.create({
  carte: {
    padding: 16,
    borderRadius: 12,
    backgroundColor: '#fff',
  },
});
```

Les règles fondamentales

- Objets JavaScript, propriétés en **camelCase**
- Valeurs numériques = points de densité (pas de px)
- **Pas de cascade**, pas d'héritage (sauf Text imbriqué)
- Pas de sélecteurs, pas de media queries
- `StyleSheet.create()` optimise les styles au démarrage

Composer les styles

```
<View style=[[styles.carte, estActif && styles.carteActive]] />
```

Tableau de styles

Passer un tableau fusionne les styles de gauche à droite. Le dernier l'emporte en cas de conflit.

Valeurs falsies ignorées

Les valeurs `false`, `null` et `undefined` sont ignorées silencieusement. Parfait pour les styles conditionnels.

Éviter les styles inline

Un style inline comme `style={{ color: 'red' }}` crée un nouvel objet à chaque rendu. Préférez `StyleSheet.create()`.

Flexbox : les différences avec le web

Propriété	Web	React Native
display	block	toujours flex
flexDirection	row	column ⚠
position	static	relative
alignContent	stretch	flex-start

⚠ Le piège classique : oublier que la direction par défaut est **verticale** (column), contrairement au web où c'est row.

flex: 1 — le réflexe fondamental

```
<View style={{ flex: 1 }}>
  /* prend tout l'espace disponible */
  <View style={{ flex: 2 }} /> /* 2/3 */
  <View style={{ flex: 1 }} /> /* 1/3 */
</View>
```

À retenir absolument

- flex: 1 sur le conteneur racine = plein écran
- Les valeurs flex sont des **ratios** entre frères
- Un View sans flex ni dimensions fixes prend la taille de son contenu

Si votre écran semble vide ou coupé, commencez par vérifier que les conteneurs ont flex: 1.

Ombres et élévation

Approche moderne (RN 0.76+)

```
boxShadow: '0px 2px 8px rgba(0,0,0,0.15)'
```

Fonctionne sur web et mobile. C'est la syntaxe CSS standard — la plus simple à utiliser.

Approche historique

```
shadowColor: '#000',  
shadowOffset: { width: 0, height: 2 },  
shadowOpacity: 0.15,  
shadowRadius: 8,  
elevation: 4, // Android uniquement
```

`elevation` est ignoré sur le web. Nécessaire pour cibler Android spécifiquement.

Responsive avec useWindowDimensions

```
import { useWindowDimensions } from 'react-native';  
  
const { width } = useWindowDimensions();  
const colonnes = width > 700 ? 3 : width > 500 ? 2 : 1;
```

Le hook se met à jour automatiquement à la rotation de l'écran **et** au redimensionnement du navigateur — parfait pour tester différentes tailles sans device physique.

 Voir EX-17 pour un exemple de grille responsive utilisant ce hook avec FlatList et numColumns.

Design tokens : votre fichier theme.ts

```
export const theme = {  
  couleurs: {  
    primaire: '#2563eb',  
    texte: '#111827',  
    fond: '#f9fafb'  
  },  
  espacements: {  
    xs: 4, sm: 8, md: 16,  
    lg: 24, xl: 32  
  },  
  rayons: {  
    sm: 6, md: 12, lg: 20  
  },  
};
```

Pourquoi c'est indispensable

- Cohérence visuelle garantie à l'échelle
- Modification globale en un seul endroit
- Base pour le mode sombre (swap de couleurs)
- Intégration TypeScript avec autocomplétion

Un composant qui code une couleur en dur (#2563eb) directement dans son style est une dette technique.

FlatList : le bon outil pour les listes

```
<FlatList
  data={articles}
  keyExtractor={({item}) => item.id}
  renderItem={({item}) => (
    <Carte article={item} />
  )}
  ItemSeparatorComponent={() => (
    <View style={styles.trait} />
  )}
  ListEmptyComponent={}<Vide />
  ListHeaderComponent={}<Entete />
/>
```

Pourquoi pas .map() ?

ScrollView + .map() → 1 000 éléments = 1 000 composants montés simultanément.

FlatList → **virtualisation** : seuls les éléments visibles (plus une marge tampon) sont rendus. Mémoire constante, démarrage rapide.

 Voir EX-06 pour la FlatList complète avec pagination.

Les props FlatList à connaître

Prop	Rôle
keyExtractor	Identité stable de chaque ligne (jamais l'index)
ListEmptyComponent	Affiché quand data est vide
ItemSeparatorComponent	Séparateur entre items (sauf le dernier)
onRefresh + refreshing	Tirer pour rafraîchir (pull-to-refresh)
onEndReached	Déclenchée en approchant la fin → pagination infinie
numColumns	Grille multi-colonnes

Optimiser une FlatList

1 Mémoïser renderItem

Extraire le composant ligne hors du JSX et l'envelopper avec `React.memo` pour éviter les re-rendus inutiles.

3 getItemLayout pour hauteur fixe

Si vos items ont une hauteur constante, `getItemLayout` évite les calculs de layout à chaque scroll.

2 Pas de fonctions inline

Une fonction inline dans `renderItem` est recréée à chaque rendu de la liste. Utilisez `useCallback`.

4 FlashList pour les très longues listes

La bibliothèque de Shopify est un remplacement drop-in de `FlatList`, jusqu'à 10× plus performant sur de grands volumes.

TextInput : la saisie utilisateur

```
const [email, setEmail] = useState("");

<TextInput
  value={email}
  onChangeText={setEmail}
  placeholder="votre@email.fr"
  keyboardType="email-address"
  autoCapitalize="none"
  autoCorrect={false}
  style={styles.champ}
/>
```

Types de clavier

default

Texte libre

email-address

@ visible

numeric

Pavé numérique

phone-pad

Téléphone

decimal-pad

Montants



`onChangeText` reçoit directement la chaîne, pas un événement DOM.

Un formulaire complet

État contrôlé

Un `useState` par champ, ou un objet unique pour les grands formulaires

Validation double

À la soumission ET au `onBlur` pour un feedback immédiat

Messages d'erreur

Affichés sous chaque champ, jamais dans une alerte globale

Bouton désactivé

Pendant l'envoi, pour éviter les soumissions multiples

Accessibilité

`accessibilityLabel` sur chaque champ pour les lecteurs d'écran



Voir EX-05 pour le formulaire complet avec validation.

Aller plus loin : react-hook-form + zod

La combinaison gagnante

```
const schema = z.object({  
  email: z.string().email('Email invalide'),  
  motDePasse: z.string().min(8,  
    'Au moins 8 caractères'),  
});
```

Pourquoi cette stack ?

- Validation typée avec inférence TypeScript automatique
- Beaucoup moins de re-rendus qu'avec useState
- Messages d'erreur centralisés dans le schéma
- Gestion native des formulaires imbriqués
- Intégration directe avec les composants natifs RN

Modal : remplacer Alert sur le web

```
<Modal
  visible={ouvert}
  transparent
  animationType="fade"
>
  <View style={styles.fond}>
    <View style={styles.boite}>
      <Text>Confirmer la suppression ?</Text>
      <Pressable
        onPress={() => setOuvert(false)}
      >
        <Text>Annuler</Text>
      </Pressable>
    </View>
  </View>
</Modal>
```

Pourquoi pas Alert.alert() ?

Sur le web, `Alert.alert()` est sans effet. Le Modal est votre remplacement universel, entièrement stylisable.

Recette

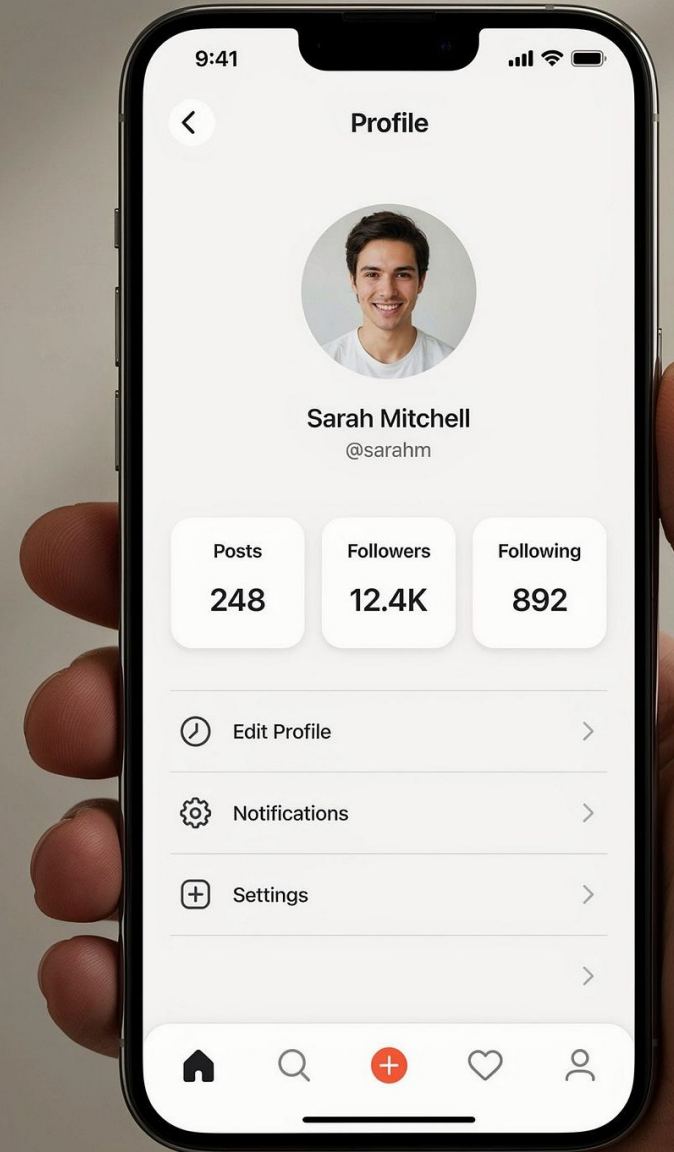
- `transparent` pour un fond semi-opaque
- `animationType="fade"` ou `"slide"`
- Fond cliquable pour fermer

 Voir EX-08 pour le Modal de confirmation complet.

PARTIE 3 — NAVIGATION, DONNÉES, ÉTAT

Connecter votre application

Navigation multi-écrans, consommation d'API, gestion de l'état et persistance locale.



Le problème de la navigation mobile



Pile d'écrans

Historique de navigation avec geste de retour natif — swipe de gauche sur iOS, bouton retour sur Android.



Onglets persistants

Des sections principales accessibles à tout moment, avec état préservé entre les visites.



Transitions animées

Les animations de navigation font partie de l'expérience native — trop souvent négligées.



Deep links

Chaque écran doit être accessible via une URL partageable, sur web comme sur mobile.

Expo Router : le routage par fichiers

```
app/
├─ _layout.tsx    layout racine
├─ index.tsx      → /
├─ profil.tsx     → /profil
├─ articles/
│   └─ index.tsx  → /articles
│   └─ [id].tsx   → /articles/42
└─ +not-found.tsx 404
```

Le principe

L'arborescence de fichiers **est** la navigation. Créer un fichier crée une route. Aucune configuration manuelle.

Inspiré de Next.js

Si vous connaissez le App Router de Next.js, vous connaissez déjà Expo Router. La syntaxe est intentionnellement identique.

Naviguer entre les écrans

Navigation déclarative

```
import { Link } from 'expo-router';  
  
<Link href="/articles/42">  
  Voir l'article  
</Link>
```

Navigation impérative

```
import { router } from 'expo-router';  
  
router.push('/articles/42');  
router.replace('/connexion');  
router.back();
```



Bonus web : chaque écran a une vraie URL partageable. Parfait pour les tests et les liens directs.

Lire les paramètres de route

```
// app/articles/[id].tsx
import { useLocalSearchParams } from 'expo-router';

export default function Article() {
  const { id } = useLocalSearchParams<{ id: string }>();
  return <Text>Article n°{id}</Text>;
}
```

useLocalSearchParams

Lit les paramètres de la route locale. TypeScript vous force à déclarer le type attendu.

useGlobalSearchParams

Accède aux paramètres depuis n'importe quel composant de l'arbre, même dans un layout parent.

Un layout avec onglets

```
// app/(tabs)/_layout.tsx
import { Tabs } from 'expo-router';

export default function Layout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: '#2563eb'
      }}
    >
      <Tabs.Screen
        name="index"
        options={{ title: 'Accueil' }}
      />
      <Tabs.Screen
        name="favoris"
        options={{ title: 'Favoris' }}
      />
    </Tabs>
  );
}
```

Conventions de dossiers

Notation	Effet
(dossier)	Groupe sans segment d'URL
[param]	Route dynamique
[...reste]	Route attrape-tout
_layout.tsx	Layout partagé
+not-found.tsx	Page 404

 Voir EX-09 pour un layout complet avec onglets et icônes.

Consommer une API — le pattern complet

```
useEffect(() => {  
  const ctrl = new AbortController();  
  (async () => {  
    try {  
      setChargement(true);  
      const r = await fetch(url, { signal: ctrl.signal });  
      if (!r.ok) throw new Error(`HTTP ${r.status}`);  
      setDonnees(await r.json());  
    } catch (e) {  
      if (e.name !== 'AbortError') setErreur(e.message);  
    } finally {  
      setChargement(false);  
    }  
  })();  
  return () => ctrl.abort();  
}, [url]);
```

❗ L'AbortController annule la requête si le composant est démonté — indispensable pour éviter les fuites mémoire. Voir EX-10.

Les quatre états d'un écran de données

1. Chargement

Squelette (skeleton) ou `ActivityIndicator`. Ne bloquez jamais l'écran sans feedback visuel.

2. Erreur

Message clair sur ce qui s'est passé + bouton « Réessayer ». Ne montrez jamais un écran blanc.

3. Vide

« Aucun résultat » accompagné d'une action utile (créer, recharger, changer les filtres).

4. Succès

Les données s'affichent. Le cas le plus simple — mais souvent le seul géré par les débutants.

 Oublier l'un des quatre états = point retiré en soutenance. C'est non négociable.

Extraire un hook personnalisé

```
export function useFetch<T>(url: string) {  
  const [etat, setEtat] = useState({  
    donnees: null,  
    chargement: true,  
    erreur: null  
  });  
  // ... logique fetch + abort  
  
  return { ...etat, recharger };  
}
```

Pourquoi extraire ?

- Logique réutilisable dans plusieurs écrans
- Testable indépendamment du composant
- JSX allégé, plus lisible
- Séparation claire entre UI et logique

Un hook = de la logique réutilisable, testable, qui vit hors du JSX.

 Voir EX-11 pour le hook useFetch complet.

TanStack Query : ce que vous n'écrirez plus

En 3 lignes

```
const { data, isLoading, error } =  
  useQuery({  
    queryKey: ['articles'],  
    queryFn: chargerArticles,  
  });
```

Ce que TanStack Query gère pour vous

- Cache et déduplication des requêtes
- Revalidation automatique en arrière-plan
- Réessais automatiques avec backoff
- Pagination et scroll infini
- Invalidation du cache après mutation

APIs publiques pour vos projets

jsonplaceholder.typicode.com

Données de test (posts, users, todos)

api.open-meteo.com

Météo sans clé

restcountries.com

Infos sur les pays

openlibrary.org

Base de données de livres

api.tvmaze.com

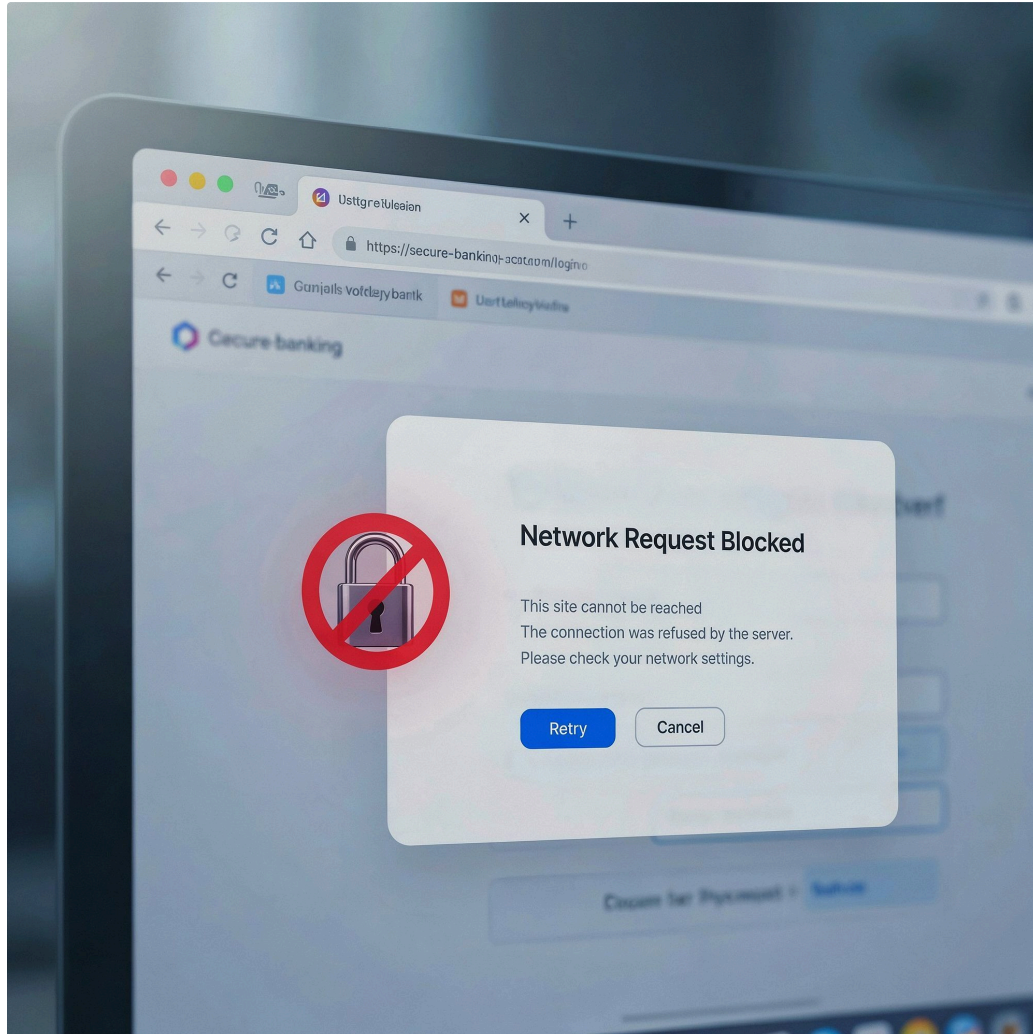
Séries TV

pokeapi.co

Pokémon

⚠ Vérifiez le CORS : en mode web, l'API doit autoriser les requêtes cross-origin. Si elle refuse, passez par un proxy local.

Le piège CORS en mode navigateur



Pourquoi ça arrive

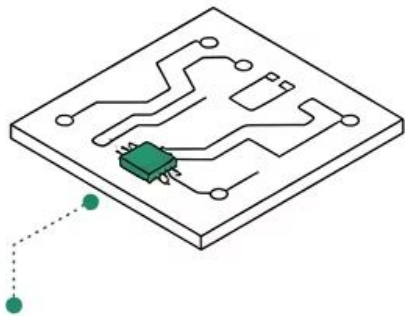
En mode navigateur, votre app est soumise à la politique d'origine croisée. Contrairement au mobile natif, le navigateur bloque les requêtes non autorisées par l'API.

Solutions

- Choisir une API qui autorise CORS
- Passer par un petit proxy Node/Express local
- Servir un fichier JSON local pendant le développement

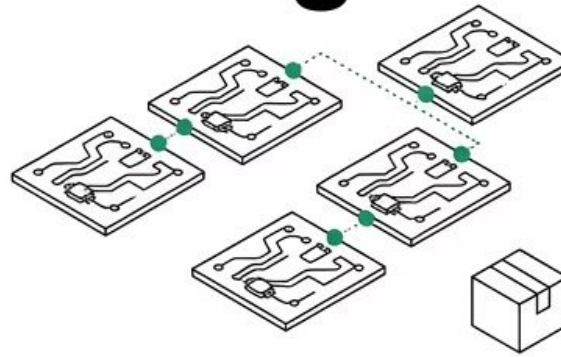
État local, partagé ou serveur ?

État Local



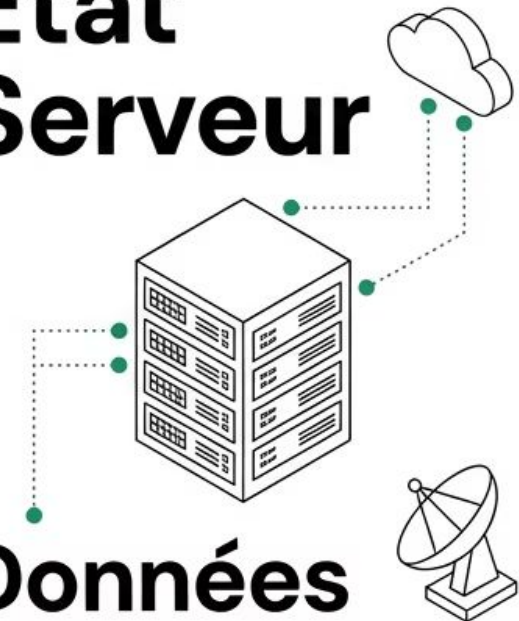
Gestion par composant
(useState, useReducer)

État Partagé



Partage entre composants
(Props, Context, Zustand)

État Serveur



Données à distance
(TanStack Query)

❏ Ne mettez pas des données serveur dans un store global — ce sont deux problèmes différents avec des solutions différentes.

Context + useReducer

```
const PanierContext = createContext(null);

export function PanierProvider({ children }) {
  const [etat, dispatch] = useReducer(
    reducer, initial
  );
  const valeur = useMemo(
    () => ({ etat, dispatch }),
    [etat]
  );
  return (
    <PanierContext.Provider value={valeur}>
      {children}
    </PanierContext.Provider>
  );
}

export const usePanier = () =>
  useContext(PanierContext);
```

Quand l'utiliser

- État partagé entre plusieurs composants éloignés
- Pas besoin de dépendance externe
- Logique de mise à jour prévisible

Limites

- Re-rendu de tous les abonnés au contexte
- Verbeux pour des besoins simples

 Voir EX-13 pour l'implémentation complète avec TypeScript.

Zustand : l'alternative légère

Un store en quelques lignes

```
const useFavoris = create((set) => ({
  ids: [],
  basculer: (id) => set((s) => ({
    ids: s.ids.includes(id)
      ? s.ids.filter(x => x !== id)
      : [...s.ids, id],
  })),
}));
```

Pourquoi Zustand ?

- Pas de Provider à envelopper
- Pas de boilerplate (actions, reducers séparés)
- Sélecteurs granulaires → re-rendus minimaux
- Persistance avec `persist middleware`
- DevTools Redux compatibles

Persistance avec AsyncStorage

```
import AsyncStorage from
  '@react-native-async-storage/async-storage';

// Écrire
await AsyncStorage.setItem(
  'favoris',
  JSON.stringify(ids)
);

// Lire
const brut = await AsyncStorage.getItem('favoris');
const favoris = brut ? JSON.parse(brut) : [];
```

À savoir

- Asynchrone, clé/valeur, chaînes uniquement
- Sur web : s'appuie sur `localStorage`
- Pas de chiffrement → ne stockez pas de secrets

Alternatives

- `expo-secure-store` pour les jetons (pas sur web)
- `expo-sqlite` pour les données relationnelles
- `MMKV` pour la performance synchrone

Le piège du chargement initial

```
const [pret, setPret] = useState(false);

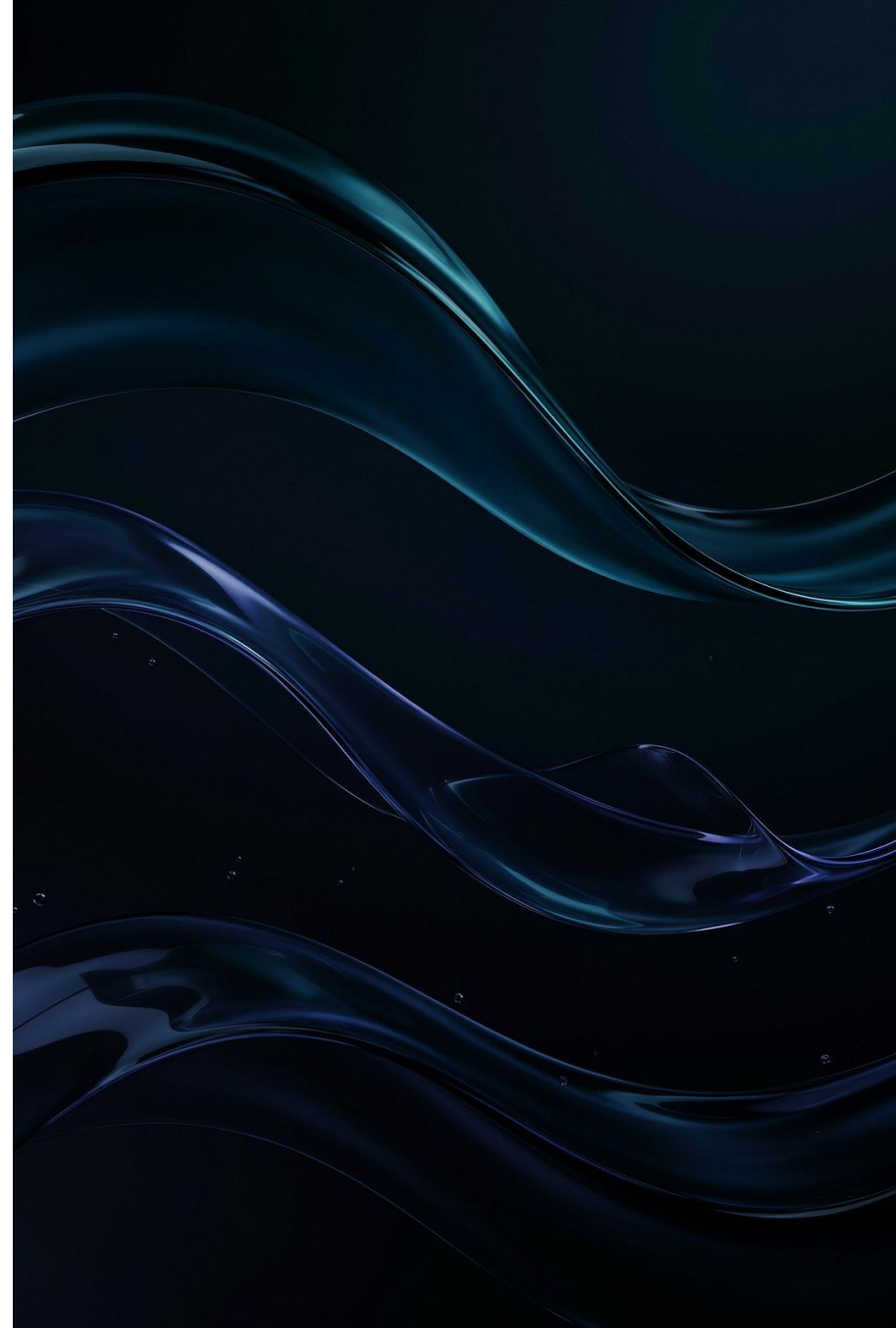
useEffect(() => {
  charger().finally(() => setPret(true));
}, []);

if (!pret) return <ActivityIndicator />;
```

⚠ La lecture AsyncStorage est asynchrone. Sans cet état `pret`, l'écran s'affiche vide une fraction de seconde puis « saute » quand les données arrivent — une mauvaise UX bien visible à la soutenance.

PARTIE 4 — ALLER PLUS LOIN

Animations, performance, qualité



Animer : l'API Animated

```
const opacity = useRef(  
  new Animated.Value(0)  
).current;  
  
useEffect(() => {  
  Animated.timing(opacity, {  
    toValue: 1,  
    duration: 400,  
    useNativeDriver: true,  
  }).start();  
}, []);  
  
<Animated.View style={{ opacity: opacity }} />
```

La règle fondamentale

Utilisez `useNativeDriver: true` dès que possible. Les animations `opacity` et `transform` s'exécutent sur le thread UI — pas sur le thread JS.

Limitations

`useNativeDriver` ne fonctionne pas pour les propriétés de layout comme `width`, `height`, `margin`. Pour ces cas, utilisez `Reanimated`.



Voir EX-14 pour des exemples d'entrée animée.

Reanimated : animations fluides à 60 fps

Le code

```
const decalage = useSharedValue(0);

const style = useAnimatedStyle(() => ({
  transform: [{
    translateX: withSpring(decalage.value)
  }],
}));
```

Pourquoi Reanimated ?

Les animations tournent entièrement sur le **thread UI**, pas sur le thread JS. Résultat : 60 fps stables même quand le thread JS est chargé.

- Animations de gestes complexes
- Transitions d'éléments partagés
- Scroll interactif et parallaxe



Voir EX-15 pour l'animation avec spring et geste.

LayoutAnimation : la solution rapide

```
import { LayoutAnimation } from 'react-native';

LayoutAnimation.configureNext(
  LayoutAnimation.Presets.easeInEaseOut
);
setElements(nouvelleListe);
```

Une seule ligne pour animer automatiquement l'apparition, la disparition ou le déplacement d'éléments dans une liste. React Native calcule les positions avant et après, et anime la transition.

 Idéal pour ajouter/supprimer des éléments dans une liste sans avoir à gérer les valeurs animées manuellement.

Accessibilité

```
<Pressable
  accessible
  accessibilityRole="button"
  accessibilityLabel="Ajouter aux favoris"
  accessibilityState={{ selected: estFavori }}
>
```

La checklist accessibilité

- Contraste suffisant (ratio 4.5:1 minimum)
- Zones tactiles de 44 points minimum
- Ne jamais coder une information par la couleur seule
- `accessibilityLabel` sur toutes les actions
- Tester avec la navigation clavier sur web

L'accessibilité n'est pas un bonus — c'est une obligation légale dans de nombreux pays.

TypeScript : typez vos props et vos API

```
type Article = {  
  id: string;  
  titre: string;  
  publieLe: string;  
  tags: string[];  
};  
  
type Props = {  
  article: Article;  
  onOuvrir: (id: string) => void;  
};  
  
export function Carte({ article, onOuvrir }: Props) {  
  /* ... */  
}
```

La règle des 80/20

Typez vos **props** et vos **réponses d'API**. Le reste du typage suit naturellement grâce à l'inférence TypeScript.

Activez le mode strict

```
// tsconfig.json  
{  
  "compilerOptions": {  
    "strict": true  
  }  
}
```

 Voir EX-19 pour les types complets de l'app exemple.

Architecture de dossiers

`src/`

- |— components/ UI réutilisable, sans logique métier
- |— features/ regroupé par domaine fonctionnel
- |— hooks/ logique partagée
- |— services/ appels API
- |— store/ état global
- |— theme/ tokens de design
- |— types/ types partagés

Le principe de découpe

Un composant qui dépasse **150 lignes** est probablement à découper. Pas parce que c'est une règle arbitraire, mais parce qu'il fait probablement trop de choses.

Feature-first vs layer-first

Pour les petits projets, par couche (components, hooks...). Pour les grands, par domaine (auth, articles, favoris...).

Tester avec Testing Library

```
import {
  render,
  screen,
  fireEvent
} from '@testing-library/react-native';

test('affiche le titre', () => {
  render(
    <Carte
      article={fixture}
      onOuvrir={jest.fn()}
    />
  );
  expect(
    screen.getByText('Mon titre')
  ).toBeTruthy();
});
```

La philosophie

Testez ce que **voit l'utilisateur**, pas l'implémentation interne. Vos tests ne doivent pas casser si vous refactorisez sans changer le comportement.

Ce qu'on teste

- Le texte visible à l'écran
- Les interactions (press, saisie)
- Les états (chargement, erreur, vide)

 Voir EX-18 pour une suite de tests complète.

Performance : les 6 réflexes

1 Virtualiser les listes

FlatList ou FlashList — jamais ScrollView + .map() pour plus de 20 éléments

2 React.memo sur les lignes

Les composants de liste ne se re-rendent que si leurs props changent

3 useCallback et useMemo avec parcimonie

Sur ce qui traverse les props vers des enfants mémorisés — pas partout

4 Images dimensionnées et mises en cache

expo-image avec des dimensions fixes — pas de chargement de mégapixels pour afficher un thumbnail

5 useNativeDriver pour les animations

Déléguer au thread UI pour des animations à 60 fps sous charge

6 Mesurer avant d'optimiser

React Native DevTools d'abord, optimisation ensuite — pas l'inverse

De l'app au store



Prébuild

EAS Build

EAS Submit

EAS Update

```
npx expo prebuild      # génère ios/ et android/  
eas build --platform all # build cloud  
eas submit            # dépôt sur les stores  
eas update            # correctif JS à distance
```

EAS Update : corriger un bug JavaScript sans repasser par la validation des stores (Apple & Google). Temps de correction : minutes, pas semaines.

Publier sur le web

```
npx expo export --platform web
```

Ce que vous obtenez

Un dossier statique (dist/) déployable en quelques minutes sur Netlify, Vercel ou GitHub Pages. Aucun serveur requis.

Le principe fondamental

Le même code source livre trois cibles.

-  Web (export statique)
-  iOS (via EAS Build)
-  Android (via EAS Build)



PARTIE 5 — L'IA DANS VOTRE PRATIQUE

Utiliser l'IA intelligemment

L'IA écrit du code React Native. Savoir juger ce code est la compétence qui vous différencie.

Le constat

L'IA écrit du code React Native correct. Elle écrit aussi du code obsolète, halluciné, ou subtilement faux. La compétence qui vous sera payée n'est plus « écrire le code ». C'est **savoir juger celui qui vous est proposé**.

Dans un monde où n'importe qui peut générer du code, la valeur réside dans la capacité à l'évaluer, le corriger, et en assumer la responsabilité.

Ce que l'IA fait bien



Code répétitif

Formulaires, listes, styles, composants
boilerplate



Conversion

Transformer du code web React en React
Native



Explication d'erreurs

Décortiquer un message d'erreur et
proposer des pistes



Tests

Écrire les tests d'un composant existant et bien défini



Documentation

JSDoc, README, commentaires explicatifs

Ce que l'IA fait mal

Les versions

Elle mélange sans vergogne les APIs de SDK 49 et SDK 57, ou utilise des props supprimées depuis 2 ans.

Les bibliothèques fantômes

Elle invente des packages qui n'existent pas, ou recommande des librairies abandonnées.

Le contexte projet

Elle ne connaît pas votre structure de dossiers, vos choix techniques, vos conventions d'équipe.

L'architecture

Les décisions structurantes demandent du jugement humain, de la connaissance du produit et de ses contraintes futures.

L'aveu d'ignorance

Elle ne dit jamais « je ne sais pas ». Elle génère toujours quelque chose qui *semble* correct.